CUESTIONARIO 12

Fechas, hilos y JVM

Pregunta 1

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuál es la salida del siguiente código?

```
import java.time.LocalDate;
import java.time.Month;

public class Test {
    public static void main(String[] argv) {
        LocalDate date = LocalDate.of(1997, Month.MAY, 50);
        System.out.println(date.getYear() + "-" + date.getMonth() + "-" + date.getDayOfMonth());
    }
}
```

Seleccione una:

- a. 1997-MAY-15
- b. 1997-JUN-15
- c. Se genera una excepción en tiempo de ejecución.
- d. El código no compila. ⊗
- e. 14-JUN-1997

Justificación: LocalDate.of valida el día del mes de forma estricta y no lo desborda al mes siguiente, así que con el día 50 lanza DateTimeException en tiempo de ejecución. La opción 1997-JUN-15 supone un ajuste automático que java.time no hace. El código compila porque 50 es un int válido.

Fuente: Simulación APX V2 #5 · también aparece en: Java.pdf #71

La respuesta correcta es: Se genera una excepción en tiempo de ejecución.

Mi justificación
El parámetro de día vale 50, no es válido porque no hay meses con 50 días, así que el código no compila.

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class Test {
    int[] array = {1, 2, 3, 4, 5};
    public static void main(String[] args) {
        new Test().go(1);
    }
    void go(int x) {
        array[x % 5] = x++;
        go(x);
    }
}
```

Seleccione una:

- a. A java.lang.ArrayIndexOutOfBoundsException is thrown.
- b. A java.lang.IllegalStateException is thrown.
- c. A java.lang.StackOverflowError is thrown. ✓
- d. The program runs with no output.
- e. Compilation fails

Justificación: go() se llama a sí mismo sin condición de parada, así que la pila se agota y se lanza StackOverflowError (verificado con JDK 21). No hay ArrayIndexOutOfBoundsException porque x % 5 da un índice entre 0 y 4 mientras x sea positivo, y la pila se desborda mucho antes de que x llegue a ser negativo.

Fuente: Java.pdf #9

La respuesta correcta es: A java.lang.StackOverflowError is thrown.

?

Mi justificación
Hacer que go(int x) recorra array[x % 5] nunca acabará de recorrerlo ya que cualquier número dará una posición válida de entre 0 y 4 y luego lo recorrerá en un ciclo infinito al ejecutarlo. Así que arroja StackOverflowError.

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
import java.util.*;
public class MyScan {
    public static void main(String[] args) {
        String in = "1 a 10 . 100 1000";
        Scanner s = new Scanner(in);
        int accum = 0;
        for (int x = 0; x < 4; x++) {
            accum += s.nextInt();
        }
        System.out.println(accum);
    }
}
```

Seleccione una:

- a. 11
- b. 111
- c. 1111
- d. An exception is thrown at runtime. ✓

Justificación: El primer nextInt() lee 1, pero el siguiente token es "a", que no es un entero, así que nextInt() lanza InputMismatchException en tiempo de ejecución. Scanner no se salta los tokens no numéricos, que es lo que suponen las opciones 11, 111 y 1111.

Fuente: Java Sesiones #11

La respuesta correcta es: An exception is thrown at runtime.

?

Mi justificación
El método s.nextInt() evalua todo el string carácter por carácter, no se salta directo a los números. Entonces al leer s.nexInt() con la primera letra del string “1 a 10 . 100 1000” tratará de evaluar con (“a”) y esto arrojaría una excepción al no ser un entero.

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

What is the result?

```
public class Bees {
    public static void main(String[] args) {
        try {
            new Bees().go();
        } catch (Exception e) {
            System.out.println("thrown to main");
        }
    }
    synchronized void go() throws InterruptedException {
        Thread t1 = new Thread();
        t1.start();
        System.out.print("1 ");
        t1.wait(5000);
        System.out.print("2 ");
    }
}
```

Seleccione una:

a. The program prints 1 then 2 after 5 seconds.

b. The program prints: 1 thrown to main. ✓

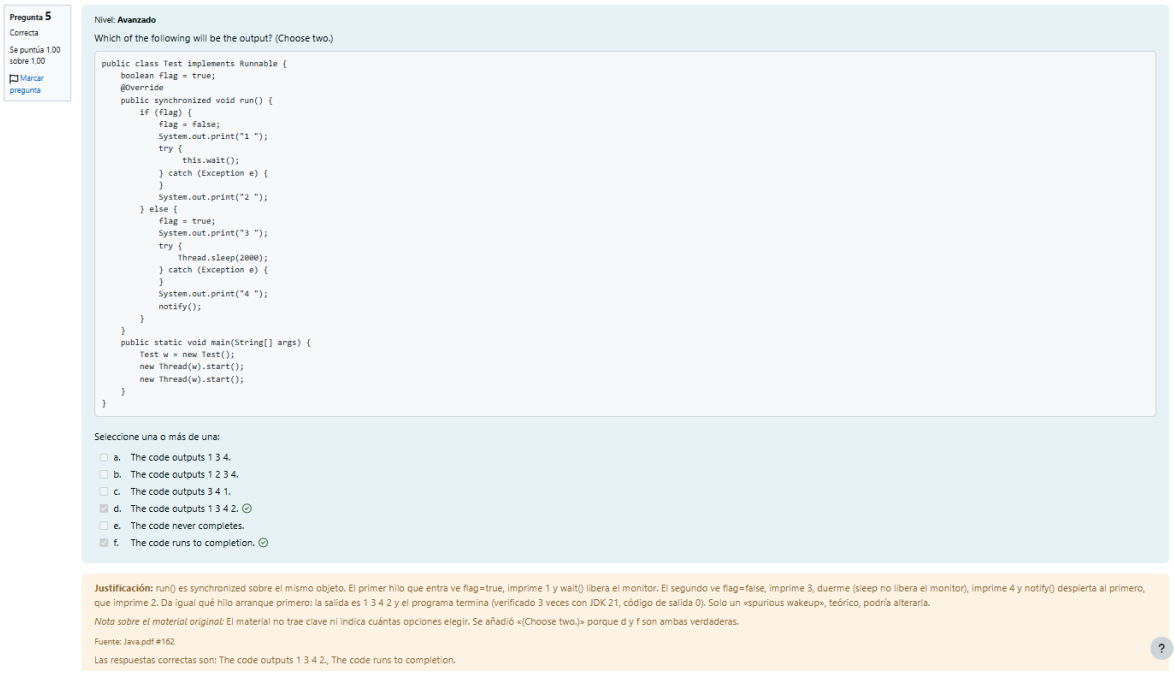
c. The program prints: 1 2 thrown to main.

d. The program prints:1 then t1 waits for its notification.

Justificación: go() es synchronized sobre this (el objeto Bees), pero llama a t1.wait() sin tener el monitor de t1. Eso lanza IllegalMonitorStateException, una excepción unchecked que atrapa el catch(Exception) de main, y por eso se imprime "1 " y luego "thrown to main". Para llamar a wait() hay que tener el bloqueo del objeto sobre el que se invoca.

Fuente: Java Sesiones #12

Mi justificación
El método go() es syncronized y no tiene al hilo t1 como parámetro, entonces no tiene su monitor y por lo tanto no puedes ejecutar wait() con él porque arroja excepción, así que solo imrpime el 1 y luego thrown to main.



Pregunta 5
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Avanzado**

Which of the following will be the output? (Choose two.)

```
public class Test implements Runnable {
    boolean flag = true;
    @Override
    public synchronized void run() {
        if (flag) {
            flag = false;
            System.out.print("1 ");
            try {
                this.wait();
            } catch (Exception e) {
            }
            System.out.print("2 ");
        } else {
            flag = true;
            System.out.print("3 ");
            try {
                Thread.sleep(2000);
            } catch (Exception e) {
            }
            System.out.print("4 ");
            notify();
        }
    }
    public static void main(String[] args) {
        Test w = new Test();
        new Thread(w).start();
        new Thread(w).start();
    }
}
```

Seleccione una o más de una:

- a. The code outputs 1 3 4.
- b. The code outputs 1 2 3 4.
- c. The code outputs 3 4 1.
- d. The code outputs 1 3 4 2. ✓
- e. The code never completes.
- f. The code runs to completion. ✓

Justificación: run() es synchronized sobre el mismo objeto. El primer hilo que entra ve flag=true, imprime 1 y wait() libera el monitor. El segundo ve flag=false, imprime 3, duerme (sleep no libera el monitor), imprime 4 y notify() despierta al primero, que imprime 2. Da igual qué hilo arranque primero: la salida es 1 3 4 2 y el programa termina (verificado 3 veces con JDK 21, código de salida 0). Solo un «spurious wakeup», teórico, podría alterarla.

Nota sobre el material original: El material no trae clave ni indica cuántas opciones elegir. Se añadió «(Choose two.)» porque d y f son ambas verdaderas.

Fuente: Java.pdf #162

Las respuestas correctas son: The code outputs 1 3 4 2., The code runs to completion.

Mi justificación
En orden: Se crea el primer hilo, flag vale true. El primer hilo entra a if(flag), cambia flag = false, imprime 1 y queda en espera por this.wait(). Se crea el segundo hilo, flag vale false. El segundo hilo entra a else, cambia flag = true, imprime 3, se pausa por 2 segundos, y luego imprime 4. Finaliza el segundo hilo. Se reanuda el primer hilo, imprime 2. Queda 1342.

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

What is the result?

```
import java.text.*;
public class Align {
    public static void main(String[] args) throws ParseException{
        String[] sa = {"111.234", "222.5678"};
        NumberFormat nf = NumberFormat.getInstance();
        nf.setMaximumFractionDigits(3);
        for (String s : sa){ System.out.println(nf.parse(s));}
    }
}
```

Seleccione una:

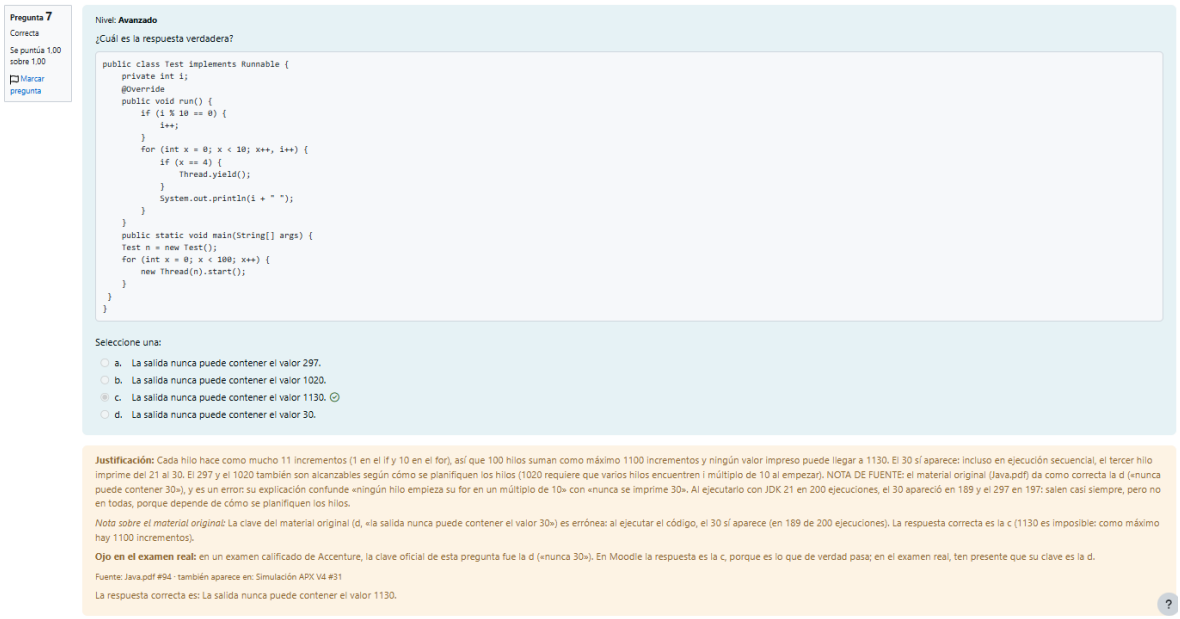
- a. 111.234 222.568
- b. 111.234 222.5678 ✓
- c. An exception is thrown at runtime.
- d. 111.234 222.567

Justificación: setMaximumFractionDigits solo afecta al formateo (format), no al análisis (parse), así que parse() devuelve el número completo: 111.234 y 222.5678. La opción a (222.568) supone que parse redondea a 3 decimales. El resultado supone un locale con punto decimal (como es_MX o en_US).

Fuente: Java DE #20 · también aparece en: Java D #1, Java Sesiones #20

La respuesta correcta es: 111.234 222.5678

Mi justificación
No conozco los métodos que se usaron, así que supuse que imprimiría lo mismo.



Pregunta 7
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Avanzado**

¿Cuál es la respuesta verdadera?

```
public class Test implements Runnable {
    private int i;
    @Override
    public void run() {
        if (i % 10 == 0) {
            i++;
        }
        for (int x = 0; x < 10; x++, i++) {
            if (x == 4) {
                Thread.yield();
            }
            System.out.println(i + " ");
        }
    }
    public static void main(String[] args) {
    Test n = new Test();
    for (int x = 0; x < 100; x++) {
        new Thread(n).start();
    }
 }
}
```

Seleccione una:

a. La salida nunca puede contener el valor 297.
b. La salida nunca puede contener el valor 1020.
c. La salida nunca puede contener el valor 1130.
d. La salida nunca puede contener el valor 30.

Justificación: Cada hilo hace como mucho 11 incrementos (1 en el if y 10 en el for), así que 100 hilos suman como máximo 1100 incrementos y ningún valor impreso puede llegar a 1130. El 30 sí aparece: incluso en ejecución secuencial, el tercer hilo imprime del 21 al 30. El 297 y el 1020 también son alcanzables según cómo se planifiquen los hilos (1020 requiere que varios hilos encuentren i múltiplo de 10 al empezar). NOTA DE FUENTE: el material original (Java.pdf) da como correcta la d («nunca puede contener 30»), y es un error: su explicación confunde «ningún hilo empieza su for en un múltiplo de 10» con «nunca se imprime 30». Al ejecutarlo con JDK 21 en 200 ejecuciones, el 30 apareció en 189 y el 297 en 197: salen casi siempre, pero no en todas, porque depende de cómo se planifiquen los hilos.

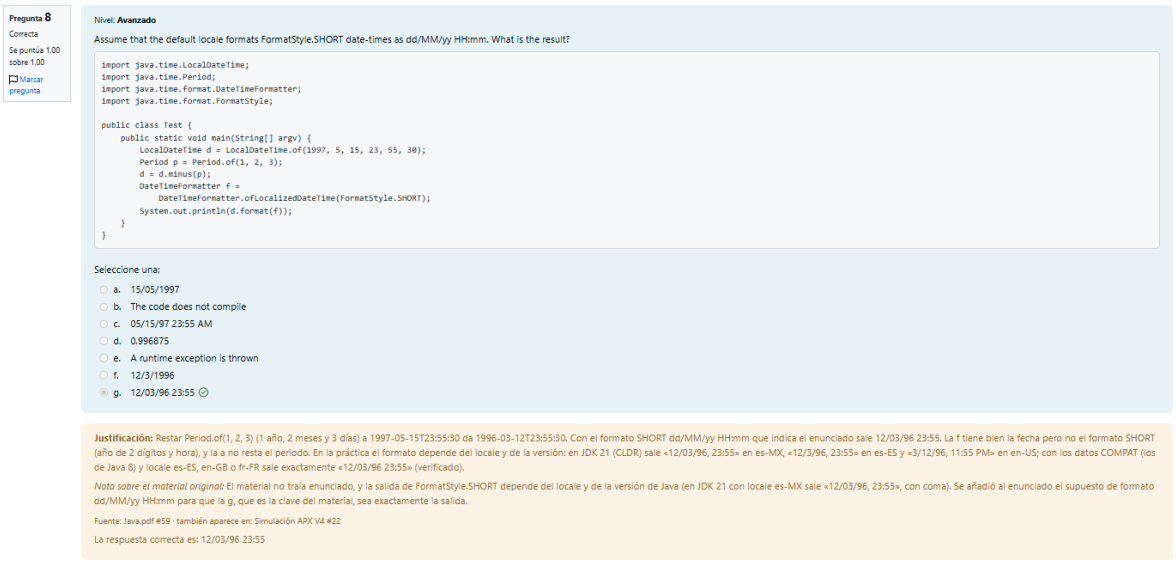
Nota sobre el material original: La clave del material original (d, «la salida nunca puede contener el valor 30») es errónea: al ejecutar el código, el 30 sí aparece (en 189 de 200 ejecuciones). La respuesta correcta es la c (1130 es imposible: como máximo hay 1100 incrementos).

Ojo en el examen real: en un examen calificado de Accenture, la clave oficial de esta pregunta fue la d («nunca 30»). En Moodle la respuesta es la c, porque es lo que de verdad pasa; en el examen real, ten presente que su clave es la d.

Fuente: Java.pdf #94 · también aparece en: Simulación APX V4 #31

La respuesta correcta es: La salida nunca puede contener el valor 1130.

Mi justificación
En cada hilo, la variable i aumenta 10 veces dentro del for, más una extra cuando llega a un múltiplo de 10, que solo hace una vez. Entonces, cada hilo aumenta i 11 veces, y como son 100 hilos, su valor máximo es 1100 aproximadamente. Entonces el valor que no puede alcanzar es 1130.



Pregunta 8
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Avanzado**

Assume that the default locale formats FormatStyle.SHORT date-times as dd/MM/yy HH:mm. What is the result?

```
import java.time.LocalDateTime;
import java.time.Period;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;

public class Test {
    public static void main(String[] argv) {
        LocalDateTime d = LocalDateTime.of(1997, 5, 15, 23, 55, 30);
        Period p = Period.of(1, 2, 3);
        d = d.minus(p);
        DateTimeFormatter f =
            DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);
        System.out.println(d.format(f));
    }
}
```

Seleccione una:

a. 15/05/1997
b. The code does not compile
c. 05/15/97 23:55 AM
d. 0.996875
e. A runtime exception is thrown
f. 12/3/1996
g. 12/03/96 23:55

Justificación: Restar Period.of(1, 2, 3) (1 año, 2 meses y 3 días) a 1997-05-15T23:55:30 da 1996-03-12T23:55:30. Con el formato SHORT dd/MM/yy HH:mm que indica el enunciado sale 12/03/96 23:55. La f tiene bien la fecha pero no el formato SHORT (año de 2 dígitos y hora), y la a no resta el periodo. En la práctica el formato depende del locale y de la versión: en JDK 21 (CLDR) sale «12/03/96, 23:55» en es-MX, «12/3/96, 23:55» en es-ES y «3/12/96, 11:55 PM» en en-US; con los datos COMPAT (los de Java 8) y locale es-ES, en-GB o fr-FR sale exactamente «12/03/96 23:55» (verificado).

Nota sobre el material original: El material no traía enunciado, y la salida de FormatStyle.SHORT depende del locale y de la versión de Java (en JDK 21 con locale es-MX sale «12/03/96, 23:55», con coma). Se añadió al enunciado el supuesto de formato dd/MM/yy HH:mm para que la g, que es la clave del material, sea exactamente la salida.

Fuente: Java.pdf #59 · también aparece en: Simulación APX V4 #22

La respuesta correcta es: 12/03/96 23:55

Mi justificación
Al restar 1 año, 2 meses y 3 días al 15 de mayo de 1997, obtenemos 12 de marzo de 1996, y en formato short el año se queda con solo 2 números ósea 96. La opción correcta es la g)

RESULTADO = 7/8 = 8.75